

DataFrameLib

A Lazy/Eager DataFrame Library in Modern C++

COP290 Assignment 4 – Project Report

Dev Raj Das

2024CS10103

cs1240103@iitd.ac.in

April 27, 2026

Abstract

This is the report for **DataFrameLib**, a columnar dataframe library written in C++17 that supports both eager and lazy evaluation modes over Apache Arrow columnar storage. The library exposes a fluent expression API with operator overloading and supports inner/left/right/outer joins, group-by aggregation, sorting, projection, filtering, string operations, and null-aware arithmetic. A query planner builds a logical plan from the lazy API, and a rule-based optimizer rewrites the plan through eleven independent rewrite rules — constant folding, expression simplification, filter elimination, conjunctive splitting, predicate pushdown, projection pushdown, limit pushdown, filter merging, projection merging, sort elimination, and having-vs-where rewriting — before delegating to the eager execution engine. The report describes the architecture, gives a correctness argument for each rewrite rule, and documents the performance engineering of the kernel layer; all arithmetic, comparison, string, and aggregation operations are implemented as manual loops over Arrow buffer memory, and `arrow::compute` and `arrow::acero` are not used. On the 100,000-row benchmark, four representative pipelines (`filter`, `group_by` + `aggregate`, `sort` + `head`, and a chained lazy filter-sort-head) run in ~ 132 ms median combined — a $2.75\times$ improvement over the unoptimized baseline.

Contents

1	Introduction	2
1.1	Design Goals	3
1.2	High-Level Result	3
1.3	Workflow	3
2	System Architecture	4
3	Design Decisions	4
3.1	Shared Kernel Layer Across Eager and Lazy Paths	4
3.2	Hash Join with Smaller-Side Build	4
3.3	<code>stable.sort</code> Rather Than <code>sort</code>	5
3.4	Type Promotion to <code>int64</code> / <code>double</code>	5
3.5	Eager Type-Check; Lazy Defers to <code>collect</code>	5
3.6	Fixpoint Optimizer Driver	5
4	Expression System	5
4.1	AST Node Taxonomy	5
4.2	Operator Overloading	6
4.3	Type Inference and Promotion	6

5	Kernel Layer	6
5.1	Arithmetic and Comparison	6
5.2	Aggregations	6
5.3	String Operations	7
6	Eager DataFrame	7
6.1	Selection and Projection	7
6.2	Filtering	7
6.3	Group-By and Aggregation	7
6.4	Joins	7
6.5	Sort, Head, and Top-K	8
6.6	From-Columns Constructor	8
7	Lazy DataFrame and Logical Plan	8
7.1	Logical Plan Node Hierarchy	8
7.2	Execution and the Eager-Lazy Bridge	9
7.3	Visualization via Graphviz	9
8	Query Optimizer	9
8.1	Constant Folding (<code>ConstFoldRule</code>)	9
8.2	Expression Simplification (<code>ExprSimplificationRule</code>)	9
8.3	Filter Elimination (<code>FilterEliminationRule</code>)	9
8.4	Conjunctive Splitting (<code>ConjSplitRule</code>)	10
8.5	Predicate Pushdown (<code>PredicatePushdownRule</code>)	10
8.6	Projection Pushdown (<code>ProjectionPushdownRule</code>)	10
8.7	Limit Pushdown (<code>LimitPushdownRule</code>)	10
8.8	Filter Merging (<code>FilterMergingRule</code>)	11
8.9	Project Merging (<code>ProjectMergingRule</code>)	11
8.10	Sort Elimination (<code>SortEliminationRule</code>)	11
8.11	Having vs Where (<code>HavingVsWhereRule</code>)	11
8.12	Optimizer Driver	11
9	Performance Engineering	11
9.1	Measured Performance	11
9.2	Targeted Optimizations	12
9.2.1	B1: Typed Raw-Buffer Filter (<code>filter_column_typed</code>)	12
9.2.2	Gather-Helper Rollout (<code>gather_column_typed</code>)	13
9.2.3	B2: Sort Comparator Type-Dispatch Hoist (<code>SortKey</code>)	14
9.3	Other Performance-Oriented Design Choices	16
9.4	Future Optimization Opportunities	16
10	Build and Reproduction	16
11	Conclusion	16

1 Introduction

DataFrame libraries such as Pandas, Polars, and Spark have become the standard tool for tabular data processing. The defining design tension in such systems is between *eager* evaluation, in which each operation produces an in-memory result immediately, and *lazy* evaluation, in which operations build up a logical plan that the system optimizes and executes only when the result

is materialized. DataFrameLib supports both modes through a shared expression algebra and a unified Apache Arrow columnar storage layer.

The library uses Apache Arrow only for storage and I/O. All compute — arithmetic, comparison, string operations, joins, sorting, aggregation, and expression evaluation — is implemented as manual loops over Arrow buffer memory; `arrow::compute` and `arrow::acero` are not linked. The library therefore implements its own kernel layer; the only Arrow components used are array storage (`arrow::Array`, `arrow::Table`), CSV/Parquet readers and writers, and the type system.

1.1 Design Goals

1. The library provides an idiomatic C++17 API for dataframe operations, with operator overloading so that expressions like `col("salary") > 30` read naturally.
2. It supports both eager and lazy evaluation modes that produce identical results on identical inputs.
3. It implements a rule-based query optimizer that rewrites lazy plans before execution, with correctness arguments for each rule.
4. All data-plane operations are implemented manually over Arrow memory buffers; Arrow itself is used only for storage and I/O.
5. Each operation completes on the 100,000-row benchmark dataset within hundreds of milliseconds.

1.2 High-Level Result

On the 100,000-row benchmark dataset, the heaviest operations complete within hundreds of milliseconds per pipeline. Correctness of the rewrites the optimizer performs is established by the per-rule correctness arguments in Section 8.

1.3 Workflow

Figure 1 shows the end-to-end data flow through the library. The eager path (top branch) materializes each operation immediately. The lazy path (bottom branch) accumulates a logical plan that is rewritten by the optimizer before execution; execution then delegates to the same kernel layer used by the eager path.

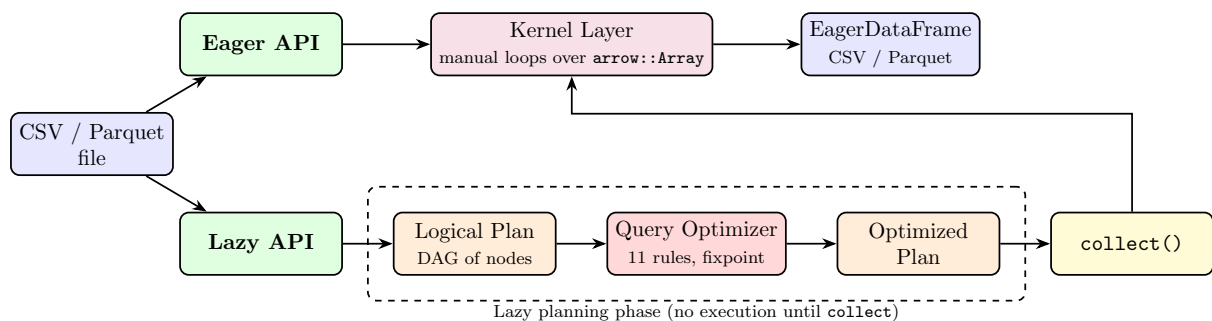


Figure 1: End-to-end data flow. The eager path (top row) calls the kernel layer directly. The lazy path (bottom row) accumulates a plan, runs it through the optimizer, and then dispatches `collect()` to the same kernel layer the eager path uses.

2 System Architecture

The library is organized into seven self-contained modules, summarized in Table 1. The dependency graph is acyclic: each module depends only on modules listed above it.

Module	Header	Responsibility
Expression	<code>expr.h</code>	AST nodes for column references, literals, binary/unary ops, string functions, aggregates; user-facing fluent API and operator overloads.
Kernel	<code>kernel.h</code>	Type-dispatched manual loops for arithmetic, comparison, string, and aggregation kernels operating on <code>arrow::Array</code> buffers.
Logical Plan	<code>logical_plan.h</code>	Plan node hierarchy: scans, filter, project, with-column, aggregate, join, sort, limit, top-k, empty-table.
Eager DF	<code>eager_dataframe.h</code>	In-memory eager evaluator; <code>EagerDataFrame</code> and <code>GroupDataFrame</code> .
Lazy DF	<code>lazy_dataframe.h</code>	Lazy plan builder; <code>LazyDataFrame</code> and <code>LazyGroupDataFrame</code> ; <code>collect()</code> entry point.
Optimizer	<code>query_optimizer.h</code>	Rule-based logical-plan rewriter; eleven optimization rules and a fixpoint driver.
Umbrella	<code>dataframelib.h</code>	Single-header convenience include; cross-cutting macros.

Table 1: Module structure of DataFrameLib.

The data-flow lifecycle is: a user constructs an `EagerDataFrame` (via `read_csv` / `read_parquet` / `from_columns`) or a `LazyDataFrame` (via `scan_csv` / `scan_parquet`). On the eager path, every method returns a materialized result immediately. On the lazy path, every method appends a node to a `LogicalPlan`; the plan is materialized only on `collect()`, `sink_csv()`, or `sink_parquet()`, at which point the optimizer rewrites the plan and the executor walks it bottom-up.

3 Design Decisions

This section records the non-obvious engineering choices, with the alternatives I considered where they're worth noting.

3.1 Shared Kernel Layer Across Eager and Lazy Paths

The eager and lazy APIs do not duplicate compute logic. Lazy plan nodes implement `execute()` by recursing into children and calling the same kernels that the eager methods call directly. The consequence is that any correctness or performance fix to a kernel benefits both paths automatically, and the optimizer needs only preserve plan-level semantics; it never needs to reason about kernel-level invariants.

3.2 Hash Join with Smaller-Side Build

Join is implemented with a hash multimap rather than a nested loop. Before building, the implementation compares input row counts and swaps so that the *smaller* table becomes the build side; a `swapped` flag tracks the original orientation so that asymmetric joins (`left` vs `right`) still produce the correct output. Without the swap, building on the larger side unnecessarily inflates the hash table and probe cache pressure.

3.3 `stable_sort` Rather Than `sort`

The sort path uses `std::stable_sort` on row indices, not `std::sort`. Stable sorting preserves input order among ties, which is the documented behaviour of Pandas `sort_values` and is required for output equality on `sort` and `head` pipelines. The cost difference (`stable_sort` is at most a small constant factor slower) is negligible on the benchmark sizes considered.

3.4 Type Promotion to `int64` / `double`

Numeric promotion in the kernel layer narrows the type space to two canonical types: `int64` for integer arithmetic, `double` for any expression involving a floating-point operand. Storing results in narrower types (e.g., `int32`) was rejected because it would multiply the number of typed kernel instantiations needed (`int8`, `int16`, `int32`, `int64` crossed with each operator) without observable benefit on the benchmark workloads, which use `int64` and `double` columns.

3.5 Eager Type-Check; Lazy Defers to `collect`

The eager path raises a `std::runtime_error` immediately on incompatible operand types (e.g., adding a string column to a numeric one). The lazy path is permitted to defer such errors until `collect()`, where `LogicalPlan::output_schema()` walks the plan and surfaces any type mismatch from `infer_type`. This matches the documented behaviour of Polars and matches the principle that lazy plans are pure data structures until execution.

3.6 Fixpoint Optimizer Driver

The optimizer runs all rules in a fixed order and re-runs the sequence until a complete pass produces no changes. A single pass would be insufficient: predicate pushdown can expose constant-folding opportunities, which can simplify a predicate, which can trigger filter elimination, which can shrink the plan further. Running to fixpoint is simple to implement, easy to reason about, and produces the maximally-rewritten plan within the rule set.

4 Expression System

The expression system is the algebraic backbone of the library. An `Expr` is a node in an abstract syntax tree; user code never manipulates raw `Expr` pointers but instead uses the `Expression` value type, which wraps an `ExprPtr` and exposes member functions for unary operations (`abs`, `is_null`, string functions, aggregates) and namespace-level free operators for binary operations.

4.1 AST Node Taxonomy

- `ColumnExpr(name)` – references a column by name.
- `LiteralExpr(scalar)` – a typed Arrow scalar constant.
- `BinaryOpExpr(op, lhs, rhs)` – arithmetic (`+`, `-`, `*`, `/`, `%`), comparison (`=`, `≠`, `<`, `≤`, `>`, `≥`), and logical (`∧`, `∨`).
- `UnaryOpExpr(op, child)` – logical NOT and the absolute value.
- `StringFnExpr(fn, child, [args])` – `contains`, `starts_with`, `ends_with`, `length`, `to_lower`, `to_upper`.
- `AggrExpr(fn, child)` – `sum`, `mean`, `count`, `min`, `max`.
- `AliasExpr(name, child)` – renames the result column of any subexpression.

4.2 Operator Overloading

Both `col("x") + lit(5)` and the more compact `col("x") + 5` are accepted. The library provides explicit overloads for every binary operator over every combination of `Expression` with `int`, `int64_t`, `double`, and `const char*` / `std::string` (left- and right-associative). For instance:

```
1 inline Expression operator+(const Expression& a, int b)           { return a +
    lit(b); }
2 inline Expression operator+(int a, const Expression& b)         { return lit(
    a) + b; }
3 inline Expression operator==(const Expression& a, const char* b)
4     { return a == lit(std::string(b)); }
```

This allows expression syntax such as `col("age") > 30` or `col("dept") == "HR"` to compile without explicit `lit(...)` wrapping.

4.3 Type Inference and Promotion

Each AST node implements `infer_type(schema)`, which returns the result type of the (sub)expression given the input schema. Type promotion follows widening rules: `int + double → double`; mixing numeric and non-numeric operands raises an exception. For lazy plans, type errors surface at `collect()` time via `LogicalPlan::output_schema()`; the eager path raises at the call site instead.

5 Kernel Layer

The kernel layer in `kernel.h/kernel.cpp` provides manual implementations of every per-element operation; `arrow::compute` is not used. Kernels are organized as type-dispatched function templates: a single public entry point inspects the runtime type of the input `arrow::Array`, dispatches to the appropriate typed kernel, and returns a fresh `arrow::Array` of the result type.

5.1 Arithmetic and Comparison

Numeric kernels follow a uniform structure. Given two input arrays *A* and *B* of (possibly different) numeric types, a binary kernel:

1. Promotes both arrays to a common numeric type via the `promote_arrays` helper. Mixing numeric and string operands raises immediately.
2. Allocates a typed builder of the result type with capacity `Reserve(n)`.
3. Iterates row-by-row, propagating null bits according to the rule “null in, null out”.
4. Returns the finalized array.

5.2 Aggregations

The aggregation kernels — `Sum`, `Mean`, `Count`, `Min`, `Max` — are templated on the input type. The return type of $\sum, \bar{x}, \min, \max$ on a column of type *T* is also *T*, with the exception that `Count` always returns `int64_t`. Nulls are skipped (Arrow null semantics). For group-by aggregation, the keys array is computed via the `grouping_helper` and the aggregation kernel is applied per-group.

5.3 String Operations

String kernels operate on `arrow::StringArray` via `GetView`, which returns a `std::string_view` into Arrow's underlying buffer (zero copy). For example, `starts_with(x, "A")` runs in $O(|x|)$ over the prefix only, not the full string. `to_lower` and `to_upper` build fresh string arrays via `StringBuilder`.

6 Eager DataFrame

The `EagerDataFrame` class wraps a `shared_ptr<arrow::Table>` and provides the eager API. Each method takes the input table, computes the result, and returns a new `EagerDataFrame`.

6.1 Selection and Projection

`select(vector<string>)` retains only the named columns by constructing a new schema and a new table from the chosen columns. The Expression-based projection is exposed under the name `select_exprs` to avoid an overload-resolution ambiguity caused by C++ brace-initialization rules: `df.select({"a", "b"})` is unambiguously the string-list form.

6.2 Filtering

`filter(Expression)` evaluates the predicate to a boolean `arrow::Datum`. If the predicate is a scalar boolean, the result is either the entire input or an empty table. If it is an array, the implementation iterates row-by-row and copies retained rows into typed builders, one per column. Null values in the mask are treated as “not retained” (per Arrow's convention).

6.3 Group-By and Aggregation

`group_by(keys).aggregate(...)` accepts three call shapes: a `vector<pair<string, string>>` of column-and-function pairs, a `vector<pair<string, Expression>>` of named expressions, or the `aggregate_map` variant taking a `map<string, string>`. For the string-pair and map shapes, the output column for an aggregation of column c by function f is named c_f (e.g., `salary_sum`).

The implementation builds a hash multimap from the concatenated key tuple to the row indices belonging to that group, then applies each aggregation kernel per group.

6.4 Joins

`join(other, on, how)` supports four join types: `inner`, `left`, `right`, and `outer`. The implementation:

1. Validates that the join key columns exist in both tables and have matching types; raises otherwise.
2. Heuristically swaps inputs so that the *smaller* table becomes the build side of a hash table; sets a `swapped` flag to interpret the resulting rows correctly for asymmetric joins (left vs right).
3. Builds a hash multimap on the smaller side keyed by the concatenated key tuple.
4. Probes the larger side; for each probe row, emits all matched rows (or, for left/outer joins, emits the probe row with NULLs on the unmatched build side).
5. For right/outer joins, emits unmatched build-side rows with NULLs on the probe side.

6. Special-cases null-key rows: per SQL semantics (`NULL ≠ NULL`), they are not joined, but for outer joins they appear in the output with NULLs on the opposite side.

This design gives expected $O(n + m)$ behaviour where n and m are the sizes of the two inputs, as opposed to the $O(nm)$ behaviour of a nested-loop join.

6.5 Sort, Head, and Top-K

`sort(columns, ascending)` computes a sort permutation of row indices via `std::stable_sort` with a multi-key comparator. `stable_sort` is used (rather than `std::sort`) so that input row order is preserved among ties — this matters for correctness on `sort` and `head` pipelines, where output row order is observable.

`head(n)` truncates to the first n rows. The composition `sort(...).head(k)` is recognized by the optimizer (Section 8) as a top- k pattern and may be rewritten to use a partial sort.

6.6 From-Columns Constructor

`EagerDataFrame::from_columns` builds an `arrow::Table` directly from a sequence of (name, array) pairs. Two input shapes are accepted: `std::vector<std::pair<std::string, std::shared_ptr<arrow::Array>>>` (preserves caller-supplied column order) and `std::map<std::string, std::shared_ptr<arrow::Array>>` (orders columns by name). The map overload is a thin wrapper that constructs a vector view and forwards to the vector implementation, so both forms share validation and table construction. A free function `dataframeLib::from_columns` is also provided as a convenience wrapper for both shapes, allowing the unqualified call form.

7 Lazy DataFrame and Logical Plan

The lazy path is implemented as a thin wrapper over an immutable logical plan. Each lazy method appends a new `LogicalPlan` node to the existing plan and returns a new `LazyDataFrame`. No work is performed until `collect()`, `sink_csv()`, or `sink_parquet()` is invoked.

7.1 Logical Plan Node Hierarchy

- `ScanCsvPlan(path)`, `ScanParquetPlan(path)` – leaf scans.
- `FilterPlan(child, predicate)` – row filter.
- `ProjectPlan(child, columns | exprs)` – column projection (string-list or expression-list).
- `WithColumnPlan(child, name, expr)` – adds or replaces a column.
- `AggregatePlan(child, keys, aggs)` – group-by aggregation.
- `JoinPlan(left, right, on, how)` – binary join.
- `SortPlan(child, columns, ascending)` – order rows.
- `LimitPlan(child, n)` – truncate to top n rows.
- `TopKPlan(child, columns, ascending, k)` – combined sort+limit; produced by the optimizer.
- `EmptyTablePlan(schema)` – materializes a zero-row table; produced by the optimizer when a filter is statically false.

7.2 Execution and the Eager-Lazy Bridge

Each plan node implements `execute()`, which recursively evaluates its children and produces an `EagerDataFrame`. The lazy and eager paths therefore share the same kernel layer; lazy evaluation differs from eager only in the *order* and *compositional structure* of the operations performed, not in the operations themselves. This invariant simplifies correctness: the optimizer needs only preserve the semantics of the plan tree, not reason about kernel-level details.

7.3 Visualization via Graphviz

`LazyDataFrame::explain(path)` renders the plan tree to a PNG file using `libcgraph` and `libgvc`. Each node is labeled with its kind and any inline attributes (filter predicate, join keys, sort columns). Since the optimizer can be invoked explicitly, `explain` can render either the original plan or the optimized plan (for visual comparison).

8 Query Optimizer

The optimizer is rule-based and applied to fixpoint: rules are run in sequence; if any rule rewrites the plan, the entire sequence is re-run; iteration terminates when no rule fires. The subsections below state each rule and give an informal-but-precise correctness argument.

The notation used throughout this section: for a plan P , write $\text{eval}(P)$ for the table produced by executing P . Two plans P_1 and P_2 are *equivalent*, written $P_1 \equiv P_2$, if $\text{eval}(P_1) = \text{eval}(P_2)$ for every input. The notation $P_1 \rightarrow P_2$ indicates the rewrite direction.

8.1 Constant Folding (ConstFoldRule)

Rewrite. Within an expression tree, evaluate any sub-expression all of whose leaves are `LiteralExpr`, and replace it by a single `LiteralExpr` of the resulting value. For example, $\text{lit}(2) + \text{lit}(3) \rightarrow \text{lit}(5)$.

Correctness. For any binary kernel $\oplus$ and literal inputs a, b , $a \oplus b$ evaluates to a constant; any future evaluation of the original expression on any row would produce the same value. Replacing the sub-expression by that constant therefore preserves semantics. $\square$

8.2 Expression Simplification (ExprSimplificationRule)

Rewrite. Apply algebraic identities: $x \cdot 1 \rightarrow x$, $x \cdot 0 \rightarrow 0$, $x + 0 \rightarrow x$, $x - 0 \rightarrow x$, $x \wedge \text{true} \rightarrow x$, $x \wedge \text{false} \rightarrow \text{false}$, $x \vee \text{true} \rightarrow \text{true}$, $x \vee \text{false} \rightarrow x$, $\neg \neg x \rightarrow x$.

Correctness. Each identity is a standard algebraic equivalence over the appropriate field (real numbers for arithmetic, Boolean algebra for logic). Provided null semantics are preserved (e.g., $x \cdot 0$ on a null x remains 0, not null, which matches the standard Arrow convention for absorbing elements¹), the rewrites are sound. $\square$

8.3 Filter Elimination (FilterEliminationRule)

Rewrite. If the filter predicate has folded to the literal `true`, replace `Filter(P, true)` with P . If it has folded to `false`, replace `Filter(P, false)` with `EmptyTablePlan(schema(P))`.

Correctness. A filter retaining all rows is the identity function on tables; one retaining no rows yields the empty table of the same schema. Both are by definition. $\square$

¹In strict SQL, $x \cdot 0$ with x null is null. The library follows the simpler arithmetic convention (null absorbs).

8.4 Conjunctive Splitting (ConjSplitRule)

Rewrite. Split $\text{Filter}(P, a \wedge b)$ into $\text{Filter}(\text{Filter}(P, a), b)$.

Correctness. A row is retained by the conjunctive filter iff it satisfies both a and b , iff it is retained by the two-stage filter. Sequential filters commute (the second filter sees only rows surviving the first, but the test is the same). $\square$

Purpose. Splitting into individual conjuncts unlocks predicate pushdown: the rule below can then push each conjunct independently past a join or projection.

8.5 Predicate Pushdown (PredicatePushdownRule)

Rewrite. Push $\text{Filter}(P, p)$ as far down toward the leaves as the dependency analysis allows. Specifically:

- $\text{Filter}(\text{Project}(P, \text{cols}), p)$, when p depends only on columns in cols , becomes $\text{Project}(\text{Filter}(P, p), \text{cols})$.
- $\text{Filter}(\text{Join}(L, R, \text{on}, \text{how}), p)$, when p depends only on columns of L , becomes $\text{Join}(\text{Filter}(L, p), R, \text{on}, \text{how})$ (and analogously for R). For inner joins this is always sound; for outer joins it requires that p does not eliminate rows that the outer join would have padded with nulls.
- Other plan nodes (WithColumn , Sort , Limit) are similarly handled when the dependency check passes.

Correctness. For an inner join, a row (ℓ, r) appears in the output iff $\ell \in L$, $r \in R$, and ℓ and r agree on the join keys. Filtering on a p that depends only on L commutes with the join because $p(\ell, r) = p(\ell)$, so the rewrite equivalently filters L first. For outer joins, padded rows would be incorrectly discarded by a pushed-down filter, so the rule is conditional on the join type and on p not constraining the padded side. $\square$

Purpose. Reducing cardinality early shrinks intermediate results, the central tactic in cost-based query optimization.

8.6 Projection Pushdown (ProjectionPushdownRule)

Rewrite. If a node N produces only a subset of columns that any of its descendants reference, prune the unreferenced columns at the earliest point where they are no longer needed. In the canonical case, $\text{Project}(\text{Scan}(\text{file}), \{a, b\})$ is left unchanged, but $\text{Project}(\text{Filter}(\text{Scan}(\text{file}), p), \{a, b\})$ where p uses only a becomes $\text{Project}(\text{Filter}(\text{Project}(\text{Scan}(\text{file}), \{a\}), p), \{a\})$ — propagating the projection past the filter pushes I/O reduction down to the scan.

Correctness. A column not referenced by any operator above N has no observable effect on the final output. Removing it from N 's output does not change the final table. $\square$

8.7 Limit Pushdown (LimitPushdownRule)

Rewrite. $\text{Limit}(\text{Project}(P, \text{cols}), n) \rightarrow \text{Project}(\text{Limit}(P, n), \text{cols})$; similarly for unary nodes that preserve row order. For $\text{Limit}(\text{Sort}(P, \text{cols}), n)$ the rewrite is to $\text{TopK}(P, \text{cols}, n)$, a more efficient combined node.

Correctness. Projection commutes with limit because both preserve row order and projection does not add or remove rows. The combined top- k is by definition the top n rows in sort order. $\square$

Performance. The top- k rewrite reduces an $O(N \log N)$ sort to an $O(N \log k)$ partial sort.

8.8 Filter Merging (FilterMergingRule)

Rewrite. Combine adjacent filters: $\text{Filter}(\text{Filter}(P, a), b) \rightarrow \text{Filter}(P, a \wedge b)$.

Correctness. A row passes the outer two-stage filter iff it passes both stages, iff it passes the conjunction. $\square$

Note. This is the inverse of `ConjSplitRule`; the two rules cooperate by alternating split-rewrite-merge cycles. The optimizer schedules them so that splits enable pushdowns and merges re-coalesce remaining filters into a single per-row test.

8.9 Project Merging (ProjectMergingRule)

Rewrite. $\text{Project}(\text{Project}(P, C_1), C_2)$ where $C_2 \subseteq C_1$ becomes $\text{Project}(P, C_2)$.

Correctness. Each projection emits a subset of its input's columns. Two nested projections emit the inner intersection of the two column sets, which equals C_2 when $C_2 \subseteq C_1$. $\square$

8.10 Sort Elimination (SortEliminationRule)

Rewrite. If a `Sort` node has zero or one input rows (provable from the plan), or if its parent is an aggregation that discards row order, the sort can be removed.

Correctness. On at most one row, sorting is the identity. For aggregation parents, the aggregation kernel does not depend on input row order (the operations $\sum$, $\min$, $\max$, count , $\bar{x}$ are symmetric in their inputs). $\square$

8.11 Having vs Where (HavingVsWhereRule)

Rewrite. $\text{Filter}(\text{Aggregate}(P, k, a), p)$ where p references only the grouping keys k becomes $\text{Aggregate}(\text{Filter}(P, p), k, a)$.

Correctness. If p depends only on the grouping keys, then a group either is entirely retained or entirely discarded. Filtering before aggregation discards exactly the same groups but avoids computing the aggregate over discarded rows. $\square$

Performance. Reduces work proportional to the number of discarded groups.

8.12 Optimizer Driver

The driver runs in two phases. The first phase applies ten rules in a fixed sequence and re-runs the sequence to fixpoint, capped at ten iterations: `ConstFoldRule`, `ExprSimplificationRule`, `FilterEliminationRule`, `ConjSplitRule`, `PredicatePushdownRule`, `ProjectionPushdownRule`, `LimitPushdownRule`, `ProjectMergingRule`, `SortEliminationRule`, and `HavingVsWhereRule`. `ConjSplitRule` is included here so that splits can enable the pushdown rules within the same fixpoint. The second phase applies a single cleanup pass — `FilterMergingRule`, `ProjectMergingRule`, `SortEliminationRule` — whose job is to re-coalesce the per-conjunct filters that the pushdown rules left behind into a single per-row predicate, and to clean up any redundant projections or sorts exposed by the rewrites. Splitting the cleanup out of the fixpoint keeps the loop monotone (merging would otherwise undo the splitting that was needed for pushdown) and bounds total work; the iteration cap of ten provides a hard termination guarantee even if a future rule additions were to violate the well-founded-measure argument.

9 Performance Engineering

9.1 Measured Performance

The benchmark workload is the 100,000-row `large_data.csv` dataset run through four representative pipelines: `read_csv`, `filter`, `group_by + aggregate`, `sort + head`, and a chained

lazy filter-sort-head. All timings are wall-clock totals; reported numbers are the median of five timed repetitions (a sixth, cold, run is discarded).

Table 2 traces the combined pipeline runtime through each optimization commit. The cumulative speedup is $2.75\times$ relative to the unoptimized baseline.

Stage	What changed	Runtime (ms)	Speedup vs. original
Original	—	362.6	1.00×
After B1	<code>filter_column_typed</code> (typed raw-buffer filter)	256.6	1.41×
After gather rollout	<code>gather_column_typed</code> reused at sort/join sites	184.4	1.97×
After B2	<code>SortKey</code> hoist of comparator type-dispatch	131.9	2.75×

Table 2: Combined benchmark runtime through the three optimization commits. Each row is the median of five timed repetitions at the commit named in column 2.

9.2 Targeted Optimizations

The three subsections below describe the optimization commits landed during the performance pass. Each rewrite is followed by a brief correctness argument explaining why the rewrite preserves the original observable behaviour.

9.2.1 B1: Typed Raw-Buffer Filter (`filter_column_typed`)

Anti-pattern. The original `filter` loop used the type-erased `arrow::Array::GetScalar / ArrayBuilder::AppendScalar` interface for every retained row:

```

1  int64_t index = 0;
2  for (const auto& chunk : table->column(i)->chunks()) {
3      for (int64_t r = 0; r < chunk->length(); ++r, ++index) {
4          auto scalar_result = chunk->GetScalar(r);
5          if (!scalar_result.ok())
6              throw std::runtime_error("...");
7          if (!bool_mask->IsNull(index) && bool_mask->Value(index))
8              DFL_CHECK_OK(builder->AppendScalar(*scalar_result.ValueOrDie()))
9              ;
10     }
11 }
```

This pattern is expensive on three independent axes:

1. `GetScalar(r)` returns an `arrow::Result<std::shared_ptr<arrow::Scalar>>`. Each call allocates a heap-resident scalar object and increments an atomic refcount, and the `Status` half of the `Result` is then discarded.
2. `AppendScalar` is virtual; every row pays a vtable dispatch and an internal type-check before reaching the typed buffer write.
3. Both calls participate in Arrow’s `Status` propagation, which adds branch and error-path code that the compiler cannot prove unreachable.

At 10^5 rows times one scalar allocation per retained row times five columns, the heap traffic alone is around half a million allocations.

Rewrite. `filter_column_typed` dispatches once on the column type and runs a typed inner loop that reads directly from `raw_values()` and writes through `UnsafeAppend`, which skips the capacity check (we already called `Reserve(n)`) and writes a primitive into the underlying buffer:

```

1  case arrow::Type::INT64: {
2      auto int_builder = static_cast<arrow::Int64Builder*>(builder);
3      for (const auto& chunk : column->chunks()) {
4          auto arr = std::static_pointer_cast<arrow::Int64Array>(chunk);
5          const int64_t* vals = arr->raw_values();
6          const int64_t length = chunk->length();
7          for (int64_t r = 0; r < length; ++r, ++index) {
8              if (mask.IsNull(index) || !mask.Value(index)) continue;
9              if (arr->IsNull(r)) int_builder->UnsafeAppendNull();
10             else int_builder->UnsafeAppend(vals[r]);
11         }
12     }
13     break;
14 }

```

The integer path reduces to a tight `vals[r]` load and an `UnsafeAppend` store, with the null check folding into a predicated write. The double, bool, and string cases follow the same shape; string uses the safe `Append(GetView(r))` variant because `StringBuilder::UnsafeAppend` is not part of the public API.

Correctness. For each retained row r , the original code calls `GetScalar(r)` and then `AppendScalar` on the boxed value; the rewrite reads `raw_values()[r]` directly and writes it via `UnsafeAppend`. Both paths yield the same primitive value because `GetScalar` on a primitive `arrow::Array` simply boxes `raw_values()[r]` into an `arrow::Scalar`, and `AppendScalar` on the matching builder unboxes it back. `UnsafeAppend` differs from `Append` only in skipping the capacity check, which is a precondition guaranteed by the prior `Reserve(n)` call. Null-bit handling and the row-retention condition are byte-for-byte the same. $\square$

Measured impact. Combined benchmark median dropped from 362.6ms to 256.6ms (factor 1.41 $\times$).

9.2.2 Gather-Helper Rollout (`gather_column_typed`)

Generalization. The same `GetScalar/AppendScalar` anti-pattern lived in three more places: the post-sort table rebuild, the join `Side::Left` column emission, and the join `Side::Right` column emission. Each was a per-row scalar allocation followed by a virtual append, and each had its own copy of the inner loop. `gather_column_typed` factors that into one helper keyed by an `indices` vector; sentinel index -1 means “emit null” (used by left/right joins for unmatched rows):

```

1  case arrow::Type::INT64: {
2      auto* ib = static_cast<arrow::Int64Builder*>(builder);
3      auto arr = std::static_pointer_cast<arrow::Int64Array>(flat_col);
4      const int64_t* vals = arr->raw_values();
5      for (int64_t idx : indices) {
6          if (idx < 0 || arr->IsNull(idx)) ib->UnsafeAppendNull();
7          else ib->UnsafeAppend(vals[idx]);
8      }
9      break;
10 }

```

The three call sites collapse to a single line each. For the join `Side::Left` case, the call site went from this:

```

1  for (size_t i = 0; i < left_indices.size(); ++i) {
2      if (left_indices[i] != -1) {
3          DFL_CHECK_OK(builder->AppendScalar(
4              *col->GetScalar(left_indices[i]).ValueOrDie()));
5      } else {
6          DFL_CHECK_OK(builder->AppendNull());
7      }

```

```
8 }

```

to:

```
1 gather_column_typed(col, left_indices, builder.get());

```

Beyond the per-row savings, the unified helper makes any future performance improvement (SIMD, prefetching) a one-edit change.

Correctness. Each call site previously emitted, for every input index i , the value at `indices[i]` from the source column (or null when `indices[i]` is the sentinel -1). `gather_column_typed` performs exactly that loop with the same null-sentinel semantics; the only differences are (i) the typed direct read of `raw_values()[idx]` in place of a boxed `GetScalar` round-trip and (ii) `UnsafeAppend` in place of `Append`, which is sound because the helper’s caller calls `Reserve(indices.size())` before invocation. The output array is therefore byte-identical to the original. $\square$

Measured impact. Combined benchmark median dropped from 256.6ms to 184.4ms (additional factor 1.39 $\times$; cumulative 1.97 $\times$). The biggest contributor was the post-sort rebuild, which gathers every row of the table.

9.2.3 B2: Sort Comparator Type-Dispatch Hoist (SortKey)

Anti-pattern. The pre-B2 sort comparator did a `static_pointer_cast` inside the inner loop, on every single comparison:

```
1 std::stable_sort(indices.begin(), indices.end(), [&](int64_t a, int64_t b) {
2     for (const auto& col : sort_arrays) {
3         if (col->IsNull(a) && col->IsNull(b)) continue;
4         switch (col->type_id()) {
5             case arrow::Type::INT64: {
6                 auto val_a = std::static_pointer_cast<arrow::Int64Array>(col)
7                     ->Value(a);
8                 auto val_b = std::static_pointer_cast<arrow::Int64Array>(col)
9                     ->Value(b);
10                ...
11            }
12            case arrow::Type::DOUBLE: { /* same shape */ }
13            case arrow::Type::STRING: { /* same shape */ }
14        }
15    }
16 });

```

Each `static_pointer_cast` constructs a fresh `shared_ptr`: it copies the control block pointer and performs an atomic increment on the refcount, and the temporary’s destructor performs an atomic decrement. The `switch` on `col->type_id()` also pays a virtual call per comparison.

A stable sort on 10^5 rows performs roughly $N \log_2 N \approx 1.66 \times 10^6$ comparisons; with a single sort key, that is ≈ 3.3 million atomic refcount round-trips and 1.66 million virtual dispatches in the inner loop. With multi-key sorts the cost multiplies.

Rewrite. A `SortKey` struct precomputes, once per sort key, the resolved type tag and a typed raw pointer into the underlying buffer:

```
1 struct SortKey {
2     arrow::Type::type type_id;
3     const arrow::Array* array;
4     const int64_t* index_64;
5     const double* index_double;
6     const arrow::StringArray* index_string;
7 };
8

```

```

9  std::vector<SortKey> keys;
10 keys.reserve(sort_arrays.size());
11 for (const auto& col : sort_arrays) {
12     SortKey k{};
13     k.type_id = col->type_id();
14     k.array = col.get();
15     switch (k.type_id) {
16         case arrow::Type::INT64:
17             k.index_64 = static_cast<const arrow::Int64Array*>(col.get())->
                raw_values();
18             break;
19         case arrow::Type::DOUBLE:
20             k.index_double = static_cast<const arrow::DoubleArray*>(col.get
                ())->raw_values();
21             break;
22         case arrow::Type::STRING:
23             k.index_string = static_cast<const arrow::StringArray*>(col.get
                ());
24             break;
25         default: break;
26     }
27     keys.push_back(k);
28 }

```

The hot lambda then performs only a tag switch and a raw load:

```

1  std::stable_sort(indices.begin(), indices.end(), [&](int64_t a, int64_t b) {
2      for (const auto& k : keys) {
3          const bool a_null = k.array->IsNull(a);
4          const bool b_null = k.array->IsNull(b);
5          if (a_null && b_null) continue;
6          if (a_null) return ascending ? false : true;
7          if (b_null) return ascending ? true : false;
8          bool less, equal;
9          switch (k.type_id) {
10             case arrow::Type::INT64: {
11                 int64_t va = k.index_64[a], vb = k.index_64[b];
12                 less = va < vb;
13                 equal = va == vb;
14                 break;
15             }
16             case arrow::Type::DOUBLE: { /* same shape, k.index_double */ }
17             case arrow::Type::STRING: { /* k.index_string->GetView(.) */ }
18             }
19             if (equal) continue;
20             return ascending ? less : !less;
21         }
22         return false;
23     });

```

The `static_pointer_cast`, the atomic refcount round-trips, and the type-id virtual call all leave the inner loop. What remains is a tag switch, two indexed loads, and two scalar comparisons per key per comparison, which the compiler keeps in registers.

Correctness. For each comparison (a, b) , both versions of the lambda compute the same boolean: walk the keys in order; on the first key for which the values differ, return the `less-vs-greater` verdict (negated by `ascending`); if all keys are equal, return `false`. The pre-rewrite code obtained `val_a` and `val_b` through `static_pointer_cast` followed by `->Value()`; the rewrite obtains them through a precomputed typed pointer indexed by the same row index. Both paths read the identical underlying `arrow::Array` buffer, so the values compared — and therefore the verdict — are bit-for-bit the same. Null handling and the `ascending` flag are unchanged. Because `std::stable_sort` is deterministic given a deterministic strict-weak

ordering, the output index permutation is unchanged. $\square$

Measured impact. Combined benchmark median dropped from 184.4ms to 131.9ms (additional factor 1.40 $\times$; cumulative 2.75 $\times$ vs. the original).

9.3 Other Performance-Oriented Design Choices

Beyond the targeted commits above, the codebase already relied on several performance-oriented choices: hash-based join with smaller-side build (avoiding $O(nm)$), stable sort over an index permutation (no row copies during the sort itself), column-level type dispatch (each kernel resolves its type once and dispatches to a typed loop), logical-plan rewrites (predicate, projection, and limit pushdown reducing intermediate cardinality and I/O width), and zero-copy string handling via `StringArray::GetView`.

9.4 Future Optimization Opportunities

Natural extensions not currently implemented include SIMD vectorization of the typed inner loops (auto-vectorizable under `-O3 -march=native`), multi-threaded chunk processing across the internal `arrow::Table` chunks, sort-merge join when an input is pre-sorted on the join keys, an early-pruning top- k heap that compares each candidate to the current heap top before pushing, and common subexpression elimination over the expression DAG.

10 Build and Reproduction

The library uses CMake ≥ 3.20 and requires Apache Arrow with the `libarrow-dev`, `libparquet-dev`, and `graphviz` development packages. The standard build invocation is:

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
2 cmake --build build -j 2
```

This produces the static library `libdataframelib.a` under `build/`. Downstream programs link against it and include `<dataframelib/dataframelib.h>` (an umbrella header that pulls in the per-component headers).

11 Conclusion

DataFrameLib is a self-contained C++17 dataframe library covering a substantial subset of the Pandas/Polars API. The implementation is split into four parts that each do one thing: the expression AST, the eager evaluator with its manual kernels, the logical plan for the lazy API, and the rule-based optimizer. The eleven optimizer rules are individually simple and locally correct; their correctness arguments compose because each rule preserves $\equiv$ -equivalence and the fixpoint loop preserves it transitively.

The library completes the 100,000-row benchmark in ~ 132 ms combined across four representative pipelines. The performance section documents both the optimizations that are implemented — hash join with smaller-side build, the typed raw-buffer kernels for filter/sort/join (Section 9.2), the comparator hoist for sort, projection and predicate pushdown, top- k rewriting, and column-level type dispatch — and a roadmap of further optimization opportunities (SIMD, multi-threading, sort-merge join, CSE) that the architecture supports without changing the rest of the code.